

Design Patterns and Principles

Exercise 1 & Exercise 2 — Complete Java Solutions

Deep Skilling Handbook — DotNet FSE Angular

This document provides complete, production-quality Java implementations for the Singleton Pattern and Factory Method Pattern. Each exercise includes the full project structure, all Java source files with detailed comments, UML explanation, expected output, and key design principles.

Contents:

- **Exercise 1** — Singleton Pattern (Logger Utility) — Project: SingletonPatternExample
 - **Exercise 2** — Factory Method Pattern (Document Management) — Project: FactoryMethodPatternExample
-

Exercise 1: Implementing the Singleton Pattern

Scenario: You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

What is the Singleton Pattern?

The Singleton Pattern is a **Creational Design Pattern** that restricts instantiation of a class to exactly one object. It provides a global access point to that single instance. Use cases include: logging, configuration managers, thread pools, cache, and database connection pools.

UML Structure:

Logger class — private static instance, private constructor, public static getInstance() method.

Client — calls Logger.getInstance() and uses the returned object.

Project Structure:

```
SingletonPatternExample/
├── src/
│   ├── Logger.java           (Singleton class – thread-safe, lazy init)
│   ├── LogLevel.java         (Enum for log levels)
│   └── SingletonTest.java    (Test/Main class)
└── README.md
```

File 1: LogLevel.java

```
// LogLevel.java
// Enum defining logging severity levels
```

```
public enum LogLevel {
    INFO, DEBUG, WARNING, ERROR
}
```

File 2: Logger.java (Core Singleton)

```
// Logger.java
// Singleton Pattern implementation – Thread-Safe using double-checked locking
```

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
```

```
public class Logger {

    // ■■ Step 1: Private static instance (volatile for thread visibility) ■■
    private static volatile Logger instance = null;

    // Internal state
    private int      logCount = 0;
    private String   logFilePath;
    private PrintWriter fileWriter;
    private static final DateTimeFormatter FORMATTER =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    // ■■ Step 2: Private constructor – prevents external instantiation ■■■■■■
    private Logger() {
        logFilePath = "application.log";
        System.out.println("[Logger] Logger instance CREATED (this should appear only once)");
        try {
            // Append mode = true, so logs accumulate across runs
            fileWriter = new PrintWriter(new FileWriter(logFilePath, true));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(LogLevel level, String message) {
        logCount++;
        String logLine = String.format("%s [%s] %s", LocalDateTime.now().format(FORMATTER), level, message);
        fileWriter.append(logLine + "\n");
    }
}
```


File 3: SingletonTest.java (Main / Test Class)

[illegible]

```

        Logger.getInstance().close();

        System.out.println("\n===== Test Complete =====");
    }
}

```

Expected Console Output:

```

===== Singleton Pattern – Logger Test =====

--- Test 1: Instance Identity Check ---
[Logger] Logger instance CREATED (this should appear only once)
logger1 hash: 1829164700
logger2 hash: 1829164700
logger3 hash: 1829164700
logger1 == logger2 : true
logger1 == logger3 : true

--- Test 2: Shared Logging State ---
[2025-06-15 10:00:01] [INFO   ] Application started successfully.
[2025-06-15 10:00:01] [DEBUG  ] User module initialized.
[2025-06-15 10:00:01] [WARNING] Memory usage at 75%.
[2025-06-15 10:00:01] [ERROR  ] Failed to connect to payment gateway.
Log count via logger1: 4
Log count via logger2: 4
Log count via logger3: 4

--- Test 3: Thread-Safety Verification ---
[2025-06-15 10:00:01] [INFO   ] Log from thread: Thread-A
Thread-A -> instance hash: 1829164700
[2025-06-15 10:00:01] [INFO   ] Log from thread: Thread-B
Thread-B -> instance hash: 1829164700
[2025-06-15 10:00:01] [INFO   ] Log from thread: Thread-C
Thread-C -> instance hash: 1829164700

--- Test 4: Clone Prevention ---
Clone is prevented by design (CloneNotSupportedException).

--- Logger Stats ---
Total entries logged : 7
Log file              : application.log
Instance hash code    : 1829164700
[Logger] Log file closed.

===== Test Complete =====

```

Key Design Points:

1. **private static volatile Logger instance** — volatile ensures all threads see the latest value.
2. **private Logger()** — no external class can call new Logger().
3. **Double-Checked Locking** — thread-safe without locking on every call (performance optimized).
4. **Clone prevention** — overriding clone() blocks a common workaround.
5. All three references (logger1, logger2, logger3) share the **same hash code** = same object.

How to Compile & Run:

```

# Navigate to src directory
cd SingletonPatternExample/src

# Compile all Java files
javac LogLevel.java Logger.java SingletonTest.java

```


Exercise 2: Implementing the Factory Method Pattern

Scenario: You are developing a document management system that needs to create different types of documents (Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

What is the Factory Method Pattern?

The Factory Method Pattern is a **Creational Design Pattern** that defines an interface for creating objects but lets subclasses decide which class to instantiate. It promotes the **Open/Closed Principle** — open for extension, closed for modification. Adding a new document type (e.g., CSV) requires only a new subclass, not changes to existing code.

UML Structure:

Document (interface) — open(), save(), close(), getInfo()

WordDocument, PdfDocument, ExcelDocument — implement Document

DocumentFactory (abstract) — abstract createDocument() factory method

WordDocumentFactory, PdfDocumentFactory, ExcelDocumentFactory — concrete factories

DocumentManager — client that uses factories without knowing concrete types

Project Structure:

FactoryMethodPatternExample/

■■■ src/

■ ■■■ Document.java	(Product interface)
■ ■■■ WordDocument.java	(Concrete Product 1)
■ ■■■ PdfDocument.java	(Concrete Product 2)
■ ■■■ ExcelDocument.java	(Concrete Product 3)
■ ■■■ DocumentFactory.java	(Abstract Creator)
■ ■■■ WordDocumentFactory.java	(Concrete Creator 1)
■ ■■■ PdfDocumentFactory.java	(Concrete Creator 2)
■ ■■■ ExcelDocumentFactory.java	(Concrete Creator 3)
■ ■■■ DocumentManager.java	(Client / Context class)
■ ■■■ FactoryMethodTest.java	(Test/Main class)
■■■ README.md	

File 1: Document.java (Product Interface)

```
// Document.java
// Product interface – defines the contract all document types must follow

public interface Document {

    /**
     * Opens the document for editing.
     */
    void open();

    /**
     * Saves the document to disk.
     * @param filePath the path where the file should be saved
     */
    void save(String filePath);

    /**
     * Closes the document and releases resources.
     */
    void close();

    /**
     * Returns a formatted summary of the document's properties.
     */
    String getInfo();

    /**
     * Returns the MIME type of this document format.
     */
    String getMimeType();
}
```

File 2: WordDocument.java (Concrete Product 1)

```
// WordDocument.java
// Concrete Product – Microsoft Word document (.docx)

public class WordDocument implements Document {

    private String title;
    private String author;
    private boolean isOpen;
    private int pageCount;

    public WordDocument(String title, String author) {
        this.title = title;
        this.author = author;
        this.isOpen = false;
        this.pageCount = 0;
    }

    @Override
    public void open() {
        isOpen = true;
        System.out.println("[WordDocument] Opening '" + title
            + "' in Microsoft Word...");
        System.out.println(" -> Rendering DOCX format with track-changes enabled.");
    }

    @Override
    public void save(String filePath) {
        if (!isOpen) { System.out.println("[WordDocument] Error: Document not open."); return; }
        pageCount++;
        System.out.println("[WordDocument] Saving '" + title + "' to: " + filePath + ".docx");
    }
}
```

```

        System.out.println(" -> Word XML serialization complete. Pages: " + pageCount);
    }

    @Override
    public void close() {
        isOpen = false;
        System.out.println("[WordDocument] Closing '" + title + "' - auto-save triggered.");
    }

    @Override
    public String getInfo() {
        return String.format(
            "Type: Word Document | Title: %s | Author: %s | Open: %b | Pages: %d",
            title, author, isOpen, pageCount);
    }

    @Override
    public String getMimeType() {
        return "application/vnd.openxmlformats-officedocument.wordprocessingml.document";
    }
}

```

File 3: PdfDocument.java (Concrete Product 2)

```

// PdfDocument.java
// Concrete Product - Portable Document Format (.pdf)

public class PdfDocument implements Document {

    private String title;
    private boolean isPasswordProtected;
    private boolean isOpen;
    private int version;

    public PdfDocument(String title, boolean passwordProtected) {
        this.title = title;
        this.isPasswordProtected = passwordProtected;
        this.isOpen = false;
        this.version = 17; // PDF 1.7
    }

    @Override
    public void open() {
        isOpen = true;
        System.out.println("[PdfDocument] Opening '" + title + "' in PDF viewer...");
        if (isPasswordProtected)
            System.out.println(" -> Password required. Decryption in progress...");
        System.out.println(" -> PDF " + version/10.0 + " format loaded successfully.");
    }

    @Override
    public void save(String filePath) {
        if (!isOpen) { System.out.println("[PdfDocument] Error: Document not open."); return; }
        System.out.println("[PdfDocument] Saving '" + title + "' to: " + filePath + ".pdf");
        System.out.println(" -> Flattening layers, embedding fonts, compressing streams.");
        if (isPasswordProtected)
            System.out.println(" -> Applying AES-256 encryption.");
    }

    @Override
    public void close() {
        isOpen = false;
        System.out.println("[PdfDocument] Closing '" + title + "'.");
    }

    @Override

```


File 4: ExcelDocument.java (Concrete Product 3)

```
// ExcelDocument.java
// Concrete Product – Microsoft Excel spreadsheet (.xlsx)

public class ExcelDocument implements Document {

    private String title;
    private int sheetCount;
    private boolean hasMacros;
    private boolean isOpen;

    public ExcelDocument(String title, int sheetCount, boolean hasMacros) {
        this.title = title;
        this.sheetCount = sheetCount;
        this.hasMacros = hasMacros;
        this.isOpen = false;
    }

    @Override
    public void open() {
        isOpen = true;
        System.out.println("[ExcelDocument] Opening '" + title + "' in Microsoft Excel...");
        System.out.println(" -> Loading " + sheetCount + " worksheet(s).");
        if (hasMacros)
            System.out.println(" -> VBA macros detected. Enable macros? [Yes/No]");
    }

    @Override
    public void save(String filePath) {
        if (!isOpen) { System.out.println("[ExcelDocument] Error: Document not open."); return; }
        String ext = hasMacros ? ".xlsm" : ".xlsx";
        System.out.println("[ExcelDocument] Saving '" + title
            + "' to: " + filePath + ext);
        System.out.println(" -> Calculating " + sheetCount
            + " sheet(s), saving formula cache.");
    }

    @Override
    public void close() {
        isOpen = false;
        System.out.println("[ExcelDocument] Closing '" + title + "'.");
    }

    @Override
    public String getInfo() {
        return String.format(
            "Type: Excel Document | Title: %s | Sheets: %d | Macros: %b | Open: %b",
            title, sheetCount, hasMacros, isOpen);
    }

    @Override
    public String getMimeType() {
        return hasMacros
            ? "application/vnd.ms-excel.sheet.macroenabled.12"
            : "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet";
    }
}
```

File 5: DocumentFactory.java (Abstract Creator)

```
// DocumentFactory.java
// Abstract Creator – defines the factory method and common operations

public abstract class DocumentFactory {
```

```

/**
 * FACTORY METHOD – subclasses decide which Document type to create.
 * This is the core of the Factory Method Pattern.
 * @param title    The document title
 * @return         A new Document instance
 */
public abstract Document createDocument(String title);

/**
 * Template method – uses the factory method then performs common steps.
 * This shows how the factory method integrates with a workflow.
 */
public Document createAndPrepareDocument(String title, String savePath) {
    System.out.println("\n[DocumentFactory] Creating document: '" + title + "'");

    // Step 1: Create via factory method (polymorphic)
    Document doc = createDocument(title);

    // Step 2: Open and save (common steps regardless of type)
    doc.open();
    doc.save(savePath + "/" + title.replaceAll("\\s+", "_"));

    System.out.println("[DocumentFactory] Document ready: " + doc.getInfo());
    return doc;
}

/**
 * Returns the type of document this factory creates.
 */
public abstract String getFactoryType();
}

```

File 6: WordDocumentFactory.java (Concrete Creator 1)

```
// WordDocumentFactory.java
// Concrete Creator – creates Word documents

public class WordDocumentFactory extends DocumentFactory {

    private String defaultAuthor;

    public WordDocumentFactory(String defaultAuthor) {
        this.defaultAuthor = defaultAuthor;
    }

    /**
     * Factory Method implementation – creates a WordDocument.
     */
    @Override
    public Document createDocument(String title) {
        System.out.println(" [WordDocumentFactory] Instantiating WordDocument...");
        return new WordDocument(title, defaultAuthor);
    }

    @Override
    public String getFactoryType() { return "Word Document Factory (.docx)"; }
}
```

File 7: PdfDocumentFactory.java (Concrete Creator 2)

```
// PdfDocumentFactory.java
// Concrete Creator – creates PDF documents

public class PdfDocumentFactory extends DocumentFactory {

    private boolean enablePasswordProtection;

    public PdfDocumentFactory(boolean enablePasswordProtection) {
        this.enablePasswordProtection = enablePasswordProtection;
    }

    /**
     * Factory Method implementation – creates a PdfDocument.
     */
    @Override
    public Document createDocument(String title) {
        System.out.println(" [PdfDocumentFactory] Instantiating PdfDocument...");
        return new PdfDocument(title, enablePasswordProtection);
    }

    @Override
    public String getFactoryType() { return "PDF Document Factory (.pdf)"; }
}
```

File 8: ExcelDocumentFactory.java (Concrete Creator 3)

```
// ExcelDocumentFactory.java
// Concrete Creator – creates Excel documents

public class ExcelDocumentFactory extends DocumentFactory {

    private int      defaultSheetCount;
    private boolean enableMacros;

    public ExcelDocumentFactory(int defaultSheetCount, boolean enableMacros) {
        this.defaultSheetCount = defaultSheetCount;
        this.enableMacros      = enableMacros;
    }
}
```


File 9: DocumentManager.java (Client / Context Class)

File 10: FactoryMethodTest.java (Main / Test Class)

[illegible]


```
MIME: application/vnd.ms-excel.sheet.macroenabled.12
```

```
[Manager] Closing all open documents...  
===== Test Complete =====
```

Key Design Points:

1. **Open/Closed Principle (OCP)** — to add CsvDocument, create 2 new classes only; zero changes to existing DocumentFactory, DocumentManager, or Test code.
2. **Dependency Inversion** — DocumentManager depends on Document interface, not WordDocument/PdfDocument.
3. **Single Responsibility** — each factory class has one job: create its document type.
4. **Template Method integration** — createAndPrepareDocument() in the abstract factory calls the factory method, then performs common steps (open/save) regardless of type.
5. **Polymorphism** — openDocuments is List<Document>, all operations work via interface.

How to Compile & Run:

```
# Navigate to src directory  
cd FactoryMethodPatternExample/src  
  
# Compile all files in one command  
javac Document.java WordDocument.java PdfDocument.java ExcelDocument.java \  
    DocumentFactory.java WordDocumentFactory.java PdfDocumentFactory.java \  
    ExcelDocumentFactory.java DocumentManager.java FactoryMethodTest.java  
  
# Run the test  
java FactoryMethodTest
```

Pattern Comparison: Exercise 1 vs Exercise 2

Singleton Pattern (Ex 1)

Category: Creational | Intent: One instance, global access

When to use: Shared resource (logger, config, DB pool, cache)

Participants: Singleton class only

Key mechanism: Private constructor + static getInstance() + double-checked locking

Factory Method Pattern (Ex 2)

Category: Creational | Intent: Delegate object creation to subclasses

When to use: Multiple product variants; want to decouple creation from usage

Participants: Product interface, Concrete Products, Abstract Creator, Concrete Creators

Key mechanism: Abstract createDocument() overridden in each concrete factory